

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Assessment Tool Weightage: 30%

QUESTIONS:

1. Design an Autoencoder-Based Feature Learning System

A manufacturing company wants to detect defective products from high-dimensional sensor data. Design an Undercomplete Autoencoder with suitable encoder, latent representation, decoder, activation functions, and reconstruction loss. Explain how the design performs dimensionality reduction and feature learning.

2. Design a Robust Regularized Autoencoder

A healthcare organization has noisy patient-record features and needs a robust representation for downstream classification. Design a Regularized Autoencoder using an appropriate regularization strategy. Explain the encoder-decoder structure, regularization mechanism, training objective, and how the design reduces overfitting and improves useful feature extraction.

3. Design a Deep Generative Model

A recommendation or content-generation system requires learning complex data distributions. Design a Deep Belief Network or Deep Boltzmann Machine using suitable visible and hidden layers. Illustrate the architecture, explain the learning process, and justify the choice of the model for the proposed application.

4. Design a Stochastic and Contractive Representation Model

A real-world image or signal-processing application requires representations that remain useful under small input variations. Design a model using Stochastic Encoders/Decoders or a Contractive Encoder. Specify the architecture, objective function, and robustness mechanism, and explain how the design improves representation stability.

5. Design a GAN-Based Data Generation System

An organization has limited training images and wants to generate realistic synthetic samples. Design a Generative Adversarial Network with a Generator and Discriminator. Specify the input, network flow, adversarial training process, loss functions, and evaluation approach. Compare the proposed GAN design with an Autoencoder and justify why GAN is suitable for the application.

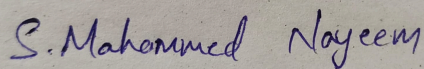
Design Desk Guidelines

- Identify the real-world problem, requirements, inputs, outputs and constraints.
- Draw a clear architecture or workflow diagram for the proposed model.
- Specify important layers, units, activation functions, latent representation and loss functions.
- Explain the training or learning mechanism and justify major design choices.
- Mention suitable evaluation measures and expected outcomes.
- Use neat labeling and present the design in a logical sequence.
- The solution should be original and written in the student's own words.

Code of Conduct:

I S.Mahammed Nayeem/19242594 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.

1. Design an Autoencoder-Based Feature Learning System

Real-World Problem

A manufacturing company collects high-dimensional sensor data such as temperature, pressure, vibration, speed, voltage, current, and acoustic measurements. Processing all sensor features directly can increase computational cost and may contain redundant information. An **Undercomplete Autoencoder (UAE)** can learn a compact representation of the sensor data and use it for detecting defective products.

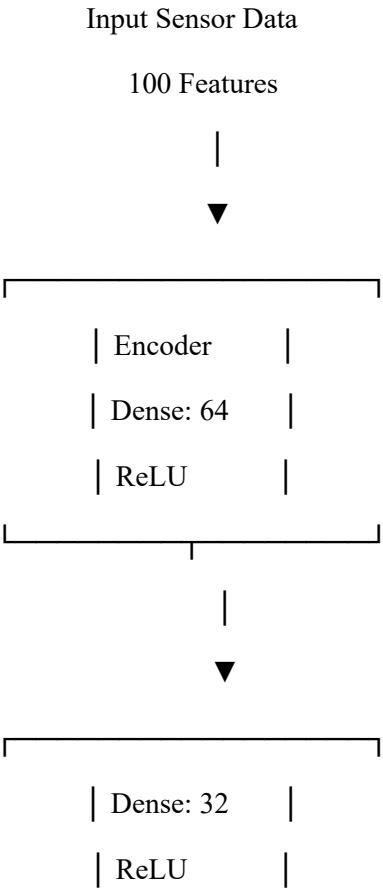
Requirements

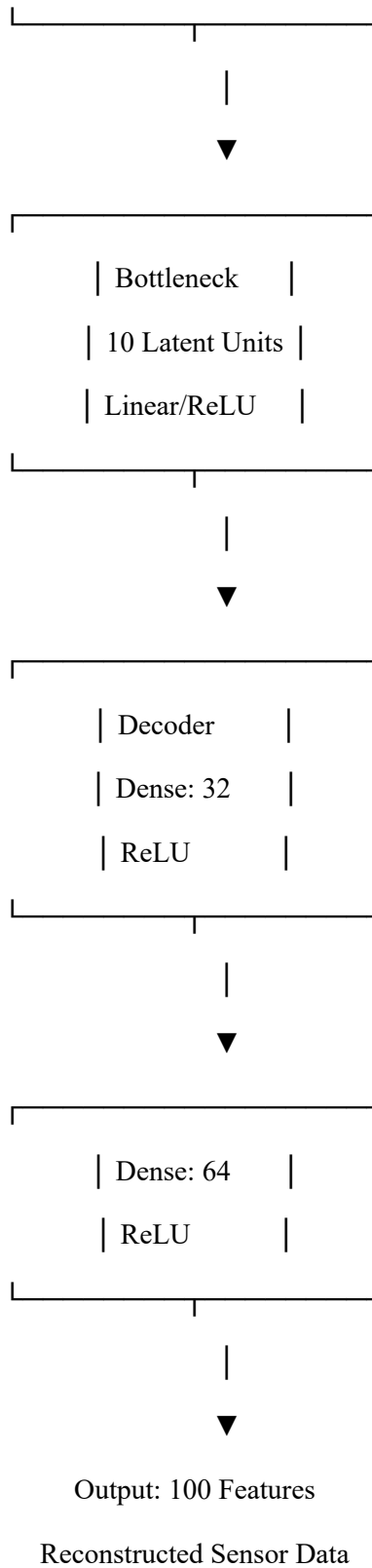
- Reduce the dimensionality of sensor data.
- Remove redundant features.
- Learn useful hidden representations automatically.
- Reconstruct normal sensor patterns accurately.
- Support downstream defect detection.

Input and Output

- **Input:** High-dimensional sensor vector, for example 100 sensor features.
- **Latent representation:** Compact vector of 10 features.
- **Output:** Reconstructed 100-dimensional sensor vector.

Proposed Architecture





Encoder

The encoder converts the original high-dimensional input into a smaller latent representation.

$$z = f_{\theta}(x) = f_{\theta}(x)$$

A suitable structure is:

- Input layer: 100 units
- Hidden layer 1: 64 units, ReLU
- Hidden layer 2: 32 units, ReLU
- Bottleneck: 10 units

Latent Representation

The 10-unit bottleneck contains the most important information required to reconstruct the original sensor data. Since 10 is much smaller than 100, the network is forced to learn meaningful and compact features.

Decoder

The decoder reconstructs the original input:

$$\hat{x} = g_{\phi}(z) = g_{\phi}(z)$$

Structure:

- 32 units, ReLU
- 64 units, ReLU
- 100 output units

For normalized continuous sensor data, a linear output activation can be used.

Activation Functions

ReLU is suitable for hidden layers because it provides nonlinear feature learning and reduces the vanishing-gradient problem. A **linear activation** is suitable for the final layer when reconstructing continuous sensor values.

Reconstruction Loss

Mean Squared Error can be used:

$$L = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

The model minimizes the difference between the original and reconstructed sensor values.

Dimensionality Reduction and Feature Learning

The undercomplete bottleneck prevents the network from simply copying the input. It must identify the most informative relationships among the sensor features. The resulting latent vector can be used as a compact feature set for a classifier such as a neural network, SVM, or random forest.

Evaluation

- Reconstruction MSE
- MAE
- Classification accuracy
- Precision, recall and F1-score for defect detection
- Latent-space visualization

Expected Outcome

The system should produce a compact sensor representation while preserving important information. Normal products should have low reconstruction error, whereas unusual or defective products may produce higher reconstruction errors.

2. Design a Robust Regularized Autoencoder

Real-World Problem

Healthcare records often contain noisy, incomplete, or inconsistent measurements. A normal autoencoder may learn unwanted noise and may overfit the training data. A **Regularized Autoencoder** can learn more useful and robust representations for downstream patient classification.

Requirements

- Handle noisy patient-record features.
- Prevent overfitting.
- Learn meaningful representations.
- Produce stable features for classification.

Proposed Model

A **Denoising Autoencoder with L2 regularization** is suitable.

Original Patient Data

|



Add Noise / Corruption

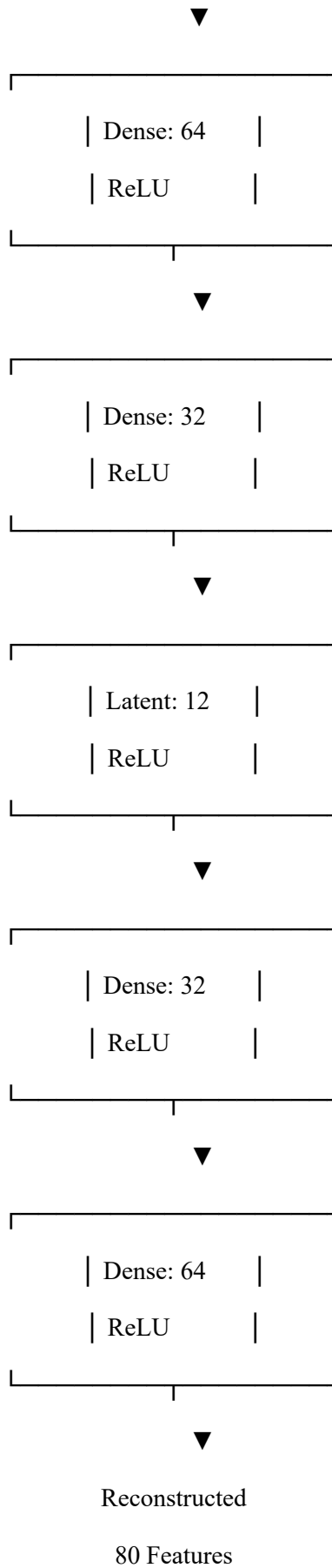
|



Noisy Input

80 Features

|



Encoder-Decoder Structure

Suppose each patient is represented by 80 numerical features.

Encoder:

- Input: 80
- Dense: 64, ReLU
- Dense: 32, ReLU
- Latent: 12, ReLU

Decoder:

- Dense: 32, ReLU
- Dense: 64, ReLU
- Output: 80, Linear

Regularization Mechanism

Two techniques can be combined:

1. **Input corruption:** Add small random noise to the input.
2. **L2 weight regularization:** Penalize very large model weights.

The objective becomes:

$$L = L_{\text{reconstruction}} + \lambda \sum W^2$$
$$L = L_{\text{reconstruction}} + \lambda \sum W^2$$

where λ controls the strength of regularization.

The noisy input is reconstructed toward the original clean input:

$$L_{\text{DAE}} = \|x - \hat{x}\|^2$$

Training Process

1. Normalize patient features.
2. Introduce controlled noise into the input.
3. Pass noisy data through the encoder.
4. Generate the latent representation.
5. Decode the representation.
6. Compare the reconstructed data with the original clean data.
7. Add the L2 regularization penalty.
8. Update network weights using backpropagation and an optimizer such as Adam.

Why It Reduces Overfitting

The model cannot simply memorize noisy training records because it receives corrupted inputs and must recover the original data. L2 regularization further discourages unnecessarily complex weight configurations.

Evaluation

- Reconstruction MSE
- Classification accuracy
- Precision
- Recall
- F1-score
- Robustness under different noise levels

Expected Outcome

The learned 12-dimensional representation should contain important patient characteristics while ignoring some irrelevant noise. This representation can then be supplied to a downstream classifier.

3. Design a Deep Generative Model

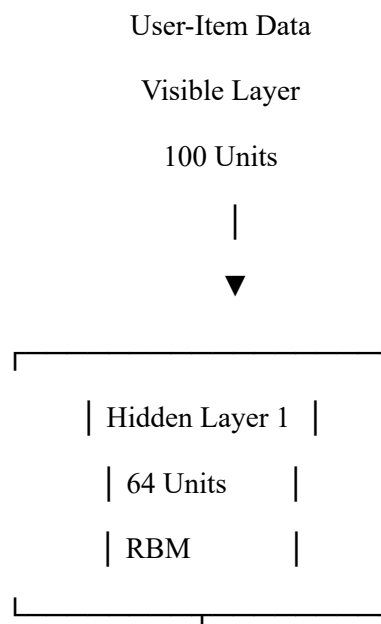
Proposed Application

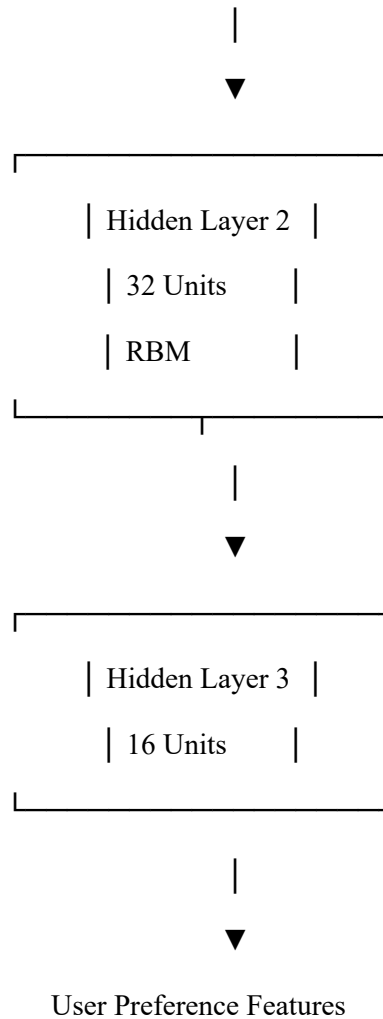
A recommendation/content-generation system needs to learn complex relationships between users and items. A **Deep Belief Network (DBN)** can learn hierarchical representations from user-item interaction data.

Input and Output

- **Input:** User-item interaction vector.
- **Output:** Learned user representation and predicted item preferences.
- **Hidden representation:** Multiple levels of increasingly abstract features.

Proposed Architecture





Structure

A DBN consists of multiple hidden layers. The lower layers learn relatively simple patterns, while deeper layers learn more abstract features.

For example:

- Visible layer: 100 user-item features
- Hidden layer 1: 64 units
- Hidden layer 2: 32 units
- Hidden layer 3: 16 units

RBM-based layers can use stochastic binary hidden units.

Learning Process

The DBN can be trained in two main stages.

Stage 1 – Layer-Wise Pretraining

Each RBM is trained separately using unsupervised learning.

Visible $\rightarrow$ Hidden 1



Hidden 2



Hidden 3

Contrastive Divergence can be used to update the weights.

Stage 2 – Fine-Tuning

After pretraining, the complete network is fine-tuned using the required recommendation objective.

RBM Energy Function

For an RBM, the energy can be expressed as:

$$E(v, h) = -v^T W h - b^T v - c^T h$$

The model learns parameters that assign lower energy to likely configurations.

Why DBN is Suitable

A DBN is useful when the data contains complex hierarchical relationships. In recommendation systems, users may have multiple levels of preferences, such as preferred category, product type, brand, and specific item.

Evaluation

- Precision@K
- Recall@K
- F1-score
- Mean Average Precision
- NDCG
- Reconstruction or likelihood-based measures

Expected Outcome

The DBN should learn compact hierarchical user and item representations and use these representations to recommend relevant content.

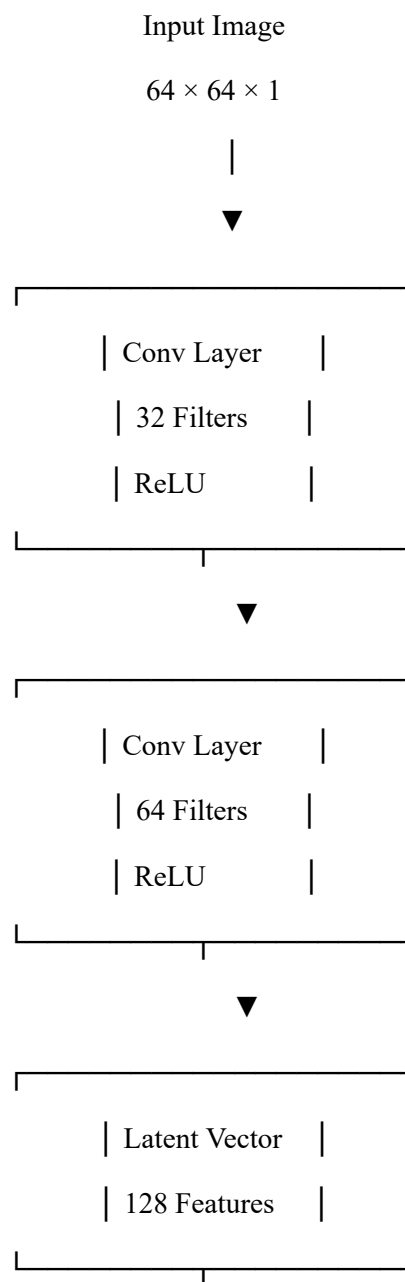
4. Design a Stochastic and Contractive Representation Model

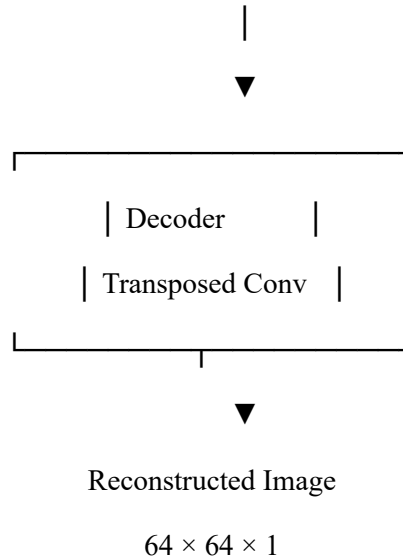
Real-World Problem

Consider an image-processing system in which images may contain small changes caused by lighting, sensor noise, rotation, or minor movement. The learned representation should remain stable under these small variations.

A **Contractive Autoencoder (CAE)** is suitable because it explicitly penalizes sensitivity of the encoder to small input changes.

Architecture





Contractive Objective

The normal reconstruction loss is:

$$L_R = ||x - \hat{x}||^2$$

A contractive penalty is added:

$$L = L_R + \lambda ||J_f(x)||_F^2$$

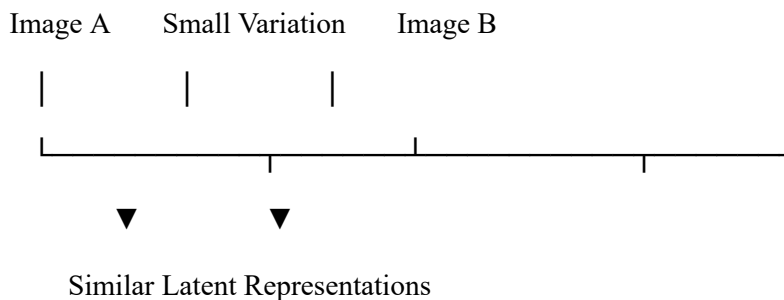
where:

- $J_f(x)$ is the Jacobian matrix of the encoder.
- $||J_f(x)||_F^2$ is the squared Frobenius norm.
- λ controls the contractive penalty.

Robustness Mechanism

The Jacobian penalty discourages the latent representation from changing significantly when the input changes slightly.

For example:



Therefore, small variations in lighting or noise should produce similar latent features.

Activation Function

ReLU can be used in the convolutional layers because it provides nonlinear feature extraction. A suitable output activation depends on image normalization. For normalized grayscale images, sigmoid can be used when values are scaled to $[0,1]$.

Training

1. Normalize input images.
2. Pass images through the encoder.
3. Generate latent representations.
4. Reconstruct the input using the decoder.
5. Calculate reconstruction loss.
6. Calculate the contractive penalty.
7. Combine both losses.
8. Update parameters through backpropagation.

Evaluation

- Reconstruction MSE
- Classification accuracy using latent features
- Robustness under Gaussian noise
- Robustness to small image transformations
- Latent-space similarity

Expected Outcome

The model should produce stable representations that preserve important image information while being less sensitive to small input variations.

5. Design a GAN-Based Data Generation System

Real-World Problem

An organization has only a limited number of training images. A **Generative Adversarial Network (GAN)** can learn the distribution of available images and generate new synthetic samples that resemble the original training data.

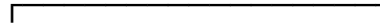
Requirements

- Generate realistic images.
- Increase the effective training dataset size.
- Preserve important visual characteristics.
- Reduce dependence on collecting large numbers of real images.

GAN Architecture

Random Noise

$$\mathbf{z} \sim \mathbf{N}(0,1)$$



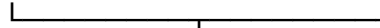
GENERATOR

Dense Layer

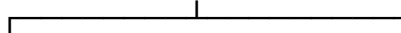
Upsampling

Conv + ReLU

Conv + Tanh



Synthetic Image

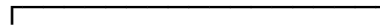
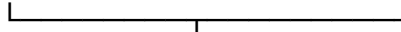


Real Training

Generated

Image

Image

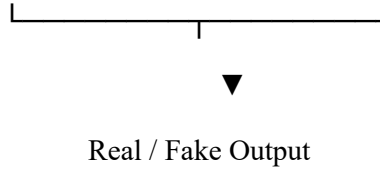


DISCRIMINATOR

Conv Layers

Leaky ReLU

Dense + Sigm



Generator

The generator receives a random noise vector.

For example:

$$z \in \mathbb{R}^{100}$$

The generator converts this random vector into an image.

Possible architecture:

- Noise input: 100
- Dense layer
- Reshape
- Transposed convolution
- Batch normalization
- ReLU
- Transposed convolution
- Batch normalization
- ReLU
- Final convolution
- Tanh output

The generator output is a synthetic image.

Discriminator

The discriminator receives either a real image or a generated image and predicts whether it is real or fake.

Possible architecture:

- Convolution layer
- Leaky ReLU
- Convolution layer
- Batch normalization
- Leaky ReLU
- Flatten
- Dense
- Sigmoid output

The output is a probability:

$$D(x) \in [0, 1]$$

where values close to 1 indicate real images.

Adversarial Training

GAN training involves two competing networks.

Generator Objective

The generator attempts to fool the discriminator:

$$L_G = -E[\log D(G(z))] \quad L_G = -E[\log D(G(z))]$$

Discriminator Objective

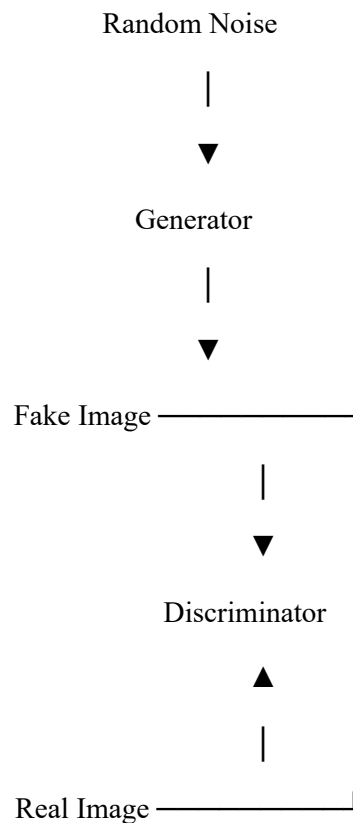
The discriminator attempts to correctly identify real and generated images:

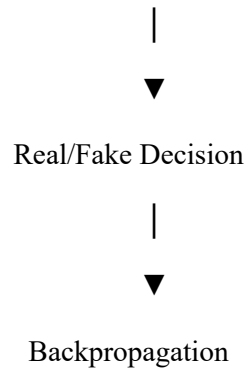
$$L_D = -E[\log D(x)] - E[\log(1 - D(G(z)))] \quad L_D = -E[\log D(x)] - E[\log(1 - D(G(z)))]$$

The overall minimax formulation is:

$$\min_G \max_D V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))] \quad \min_G \max_D V(D, G) = E[\log D(x)] + E[\log(1 - D(G(z)))]$$

Training Flow





The generator and discriminator are trained alternately. As training progresses, the generator attempts to produce increasingly realistic images, while the discriminator becomes better at distinguishing real and synthetic samples.

Evaluation

GAN-generated samples can be evaluated using:

- Visual quality
- Inception Score (IS)
- Fréchet Inception Distance (FID)
- Diversity of generated samples
- Precision and recall for generative models
- Performance of a downstream classifier trained with synthetic data

GAN vs Autoencoder

Feature	Autoencoder	GAN
Main objective	Reconstruction	Data generation
Structure	Encoder + Decoder	Generator + Discriminator
Training	Reconstruction loss	Adversarial loss
Output quality	May be smooth or blurry	Often sharper and more realistic

Latent space	Compact representation	Random generation space
Main application	Feature learning/compression	Synthetic data generation
Training difficulty	Relatively easier	More difficult and unstable
Data augmentation	Possible	Highly suitable

Why GAN is Suitable

For the given application, the main requirement is to **generate realistic additional images**, not simply reconstruct existing images. GANs are specifically designed to learn a data distribution and produce new samples. Therefore, a GAN is more suitable than a conventional autoencoder when realistic synthetic image generation is the primary goal.

Overall Design Summary

Question	Proposed Model	Main Purpose	Key Loss/Objective
1	Undercomplete Autoencoder	Dimensionality reduction	Reconstruction MSE
2	Denoising + L2 Autoencoder	Robust feature extraction	MSE + L2 penalty
3	Deep Belief Network	Deep generative representation	RBM energy / likelihood
4	Contractive Autoencoder	Stable representation	Reconstruction + Jacobian penalty
5	GAN	Synthetic image generation	Adversarial loss

Conclusion

The five designs demonstrate different applications of deep representation and generative learning. The Undercomplete Autoencoder is appropriate for dimensionality reduction and feature learning. The Regularized Autoencoder improves robustness when data contains noise. The Deep Belief Network learns hierarchical representations and complex data distributions. The Contractive Autoencoder produces representations that are stable under small input variations. Finally, the GAN is particularly suitable for generating realistic synthetic

images when training data is limited. Each architecture selects its layers, activation functions, latent representation, objective function, and evaluation metrics according to the requirements of the corresponding real-world problem.